CS 422 Assignment 2: iPerf, TCP Statistics, and Congestion Control

Daniel Chindris Saverio Miller Richie Tan Philip Wisniewski
Emma Zheng

October 2026

Repository	`https://github.com/philwisniewski/cs-422-assignment-2`
Code version	commit `d1408ba` (all code links below point at this commit)
Results used	`results/20261022-222609` (10 destinations × 3 algorithms × 3 repetitions, 60 s each)

Contents

1 Overview and reproduction **1**
1.1 Commands . . . 1
1.2 Host and run configuration . . . 2
1.3 Command-line options . . . 2
1.4 Where each requirement is implemented . . . 2

2 Q1: iPerf throughput application **4**
2.1 (a) iperf3-compatible client . . . 4
2.2 (b) Destination selection . . . 4
2.3 (c) Goodput . . . 5

3 Q2: TCP statistics during the transfer **9**
3.1 (a) Extracting TCP statistics . . . 9
3.2 (b) Visualization . . . 10
3.3 (c) Observations . . . 10

4 Q3: Comparing CUBIC, Reno and BBR, and a new algorithm **13**
4.1 (a) Experimental design . . . 13
4.2 (a) Results . . . 13
4.3 (a) Operating regimes . . . 20
4.4 (b) Designed algorithm: Gentle Reno . . . 22

5 Use of AI tools **26**

1 Overview and reproduction

Everything is driven by one entry point, `run_experiments.py`: `main`: it checks that every requested congestion-control (CC) algorithm can be selected, picks random destinations from the server list, runs

every destination × algorithm × repetition with a from-scratch Python iperf3 client (`iperf3_client.py`, the `iperf3` binary is never invoked), saves the raw per-interval `TCP_INFO` samples, and generates every table and PDF plot in this report (`analysis.py: analyze`).

1.1 Commands

```
# Server list exported from https://iperf3serverlist.net/ (in the repo as servers.csv)
sudo modprobe tcp_bbr
cat /proc/sys/net/ipv4/tcp_available_congestion_control    # must list reno cubic bbr

# Native (what produced the results in this report)
pip install -r requirements.txt
sudo python3 run_experiments.py --servers servers.csv -n 10 -t 60 -i 0.2 -r 3

# Docker (FROM ubuntu:24.04), same pipeline
docker build -t cs422-iperf .
mkdir -p data && cp servers.csv data/
docker run --rm --network host --cap-add NET_ADMIN -v "$PWD/data:/data" cs422-iperf \
    --servers /data/servers.csv --out /data/results -n 10 -t 60 -i 0.2 -r 3

# Regenerate all tables and plots from saved measurements
python3 run_experiments.py --plot-only results/20261022-222609
```

Selecting `bbr` on a socket needs `CAP_NET_ADMIN` unless it is listed in `net.ipv4.tcp_allowed_congestion_control`, hence `sudo` natively and `-cap-add NET_ADMIN` in Docker. The container shares the host kernel, so TCP behaviour and the available `TCP_INFO` fields depend on the host. Dependencies: Python ≥ 3.8, `numpy`, `matplotlib` (`requirements.txt`, `Dockerfile`).

1.2 Host and run configuration

Recorded automatically in `metadata.json` by `preflight`.

Host kernel	Linux 6.17.0-1022-nvidia (Ubuntu build), aarch64, run natively (not in Docker)
Python	3.12.3
Default CC	`cubic`
Available CC	`reno cubic bbr gentle_reno`
Allowed CC (unprivileged)	`reno cubic bbr`
Requested → actual CC (preflight)	cubic → cubic, reno → reno, bbr → bbr
Command	`python3 run_experiments.py -servers servers.csv -n 10 -t 60 -i 0.2 -r 3`
Destinations / duration / interval / reps	10 / 60 s / 0.2 s / 3
Selection seed	1396618832 (random, recorded)
Run time	2026-10-22 22:26 to 2026-10-23 00:05 EDT (1 h 39 min)

1.3 Command-line options

Option	Default	Meaning
`-servers FILE`	(required)	Server list file (addresses and ports)
`-n, -num-dests`	10	Number of random destinations
`-t, -duration`	60	Transfer duration per run, in seconds

Option	Default	Meaning
`-i, -interval`	0.2	Sampling interval for goodput and TCP_INFO, in seconds
`-r, -reps`	3	Repetitions per destination × algorithm
`-a, -algorithms`	`cubic,reno,bbr`	Algorithms to compare
`-o, -out`	`results/<timestamp>`	Output directory
`-seed`	random (recorded)	Seed for destination selection, for reproducibility
`-continent, -country`	–	Filter the server list
`-ipv4-only`	off	Skip IPv6 servers and addresses
`-max-attempts`	3·n	Maximum servers tried while selecting destinations
`-probe-duration`	3	Length of the validation test during selection
`-ports-per-server`	3	Ports of a server's range to try when one is busy or refused
`-run-retries`	2	Extra attempts for a run that fails with a retryable error
`-retry-backoff`	10	Base backoff between retries (linear, plus random jitter)
`-pause`	2	Pause between consecutive tests
`-connect-timeout / -ctrl-timeout / -stall-timeout`	5 / 15 / 15	Robustness timeouts, in seconds
`-rep-dest`	auto	Force the representative destination used for the Q2/Q3 plots
`-warmup`	0	Exclude the first N seconds of each run from summary statistics and scatter plots
`-plot-only DIR`	–	Re-analyse an existing results directory

1.4 Where each requirement is implemented

Requirement	Code
Q1(a)(i) control connection, cookie	`Iperf3Client.run` steps 1–2, `make_cookie`, `_connect`
Q1(a)(ii) JSON parameter exchange	`run` steps 3–4, `_send_json`, `_recv_json`
Q1(a)(iii) data connection	`run` steps 5–6 (CC set before `connect()` in `_connect`)
Q1(a)(iv) continuous sending	`_transfer`
Q1(a)(v) termination	`run` step 8, `_client_results`, `_abort`
Robustness	`RETRYABLE`, `Iperf3Error`, `_read_state`, `_read_server_error`, `_check_ctrl_during_test`, `run_on_server`, retries in `run_experiments`
Q1(b) destinations, replacement, shortfall	`select_destinations`, `load_servers`
Q1(c) goodput from Δbytes_acked	`_sample`
Q1(c) plot and table	`plot_goodput_all`, `build_tables`
Q2(a) `TCP_INFO` decoding	`_FIELDS`, `decode_tcp_info`, `get_tcp_info`, `get_congestion_control`
Q2(a) CSV log	`SAMPLE_COLUMNS`, `save_samples`
Q2(b) plots	`plot_timeseries`, `plot_scatter`, `make_plots`, `choose_representative`
Q3(a) runs and comparison	`run_experiments`, `set_congestion_control`, `make_plots`, `plot_regimes`, `run_stats`
Q3(b) designed algorithm	`custom_tcp.txt`, `GentleReno`, `simulate_bottleneck`, `make_design_outputs`

2 Q1: iPerf throughput application

2.1 (a) iperf3-compatible client

The client speaks the iperf3 control protocol, reverse-engineered from `src/iperf_api.c` and `iperf.h`. One test, as implemented in `Iperf3Client.run`:

```
client -> server  connect TCP control socket; send 37-byte cookie (36 chars of [a-z2-7] + NUL)
server -> client  1-byte state PARAM_EXCHANGE(9)        [busy server: ACCESS_DENIED(-1)]
client -> server  4-byte big-endian length + JSON {"tcp":true,"time":T,"parallel":1,"len":131072,...}
server -> client  CREATE_STREAMS(10)                    [rejection: SERVER_ERROR(-2) + int32 i_errno + int32 errno]
client -> server  connect data socket to the SAME resolved address (CC set first); send the same cookie
server -> client  TEST_START(1), TEST_RUNNING(2)
client -> server  bulk data for T seconds on the data socket, TCP_INFO sampled every interval
client -> server  TEST_END(4)
server -> client  EXCHANGE_RESULTS(13)
client -> server  results JSON (cpu_util_*, sender_has_retransmits, streams[{id, bytes, retransmits, ...}])
server -> client  server results JSON (includes the bytes the server received)
server -> client  DISPLAY_RESULTS(14)
client -> server  IPERF_DONE(16), close
```

The data loop (`_transfer`) is a single-threaded `select()` loop: it writes 128 KiB blocks to the non-blocking data socket until the duration elapses and wakes at every sampling deadline to read `TCP_INFO`. The client does *not* send the iperf3 `congestion` parameter: a stock server would apply it to its *receiving* socket and abort if its kernel lacked the algorithm, while only the sender's algorithm matters here.

Robustness. Every failure is raised as an `Iperf3Error(kind, phase, message)`; `RETRYABLE` decides whether it is retried.

- *Non-responsive or overloaded servers:* connect timeout (5 s), control-channel timeout (15 s) on every expected state, and a stall detector that aborts if no bytes are acknowledged for 15 s. A server answering `ACCESS_DENIED` is classified `busy`; `run_on_server` then tries other ports of the server's listed range (up to 3 ports).
- *Rejected parameter negotiation:* `SERVER_ERROR` is decoded (`i_errno`, `errno`); an error in place of `CREATE_STREAMS` is reported as `param_rejected` and not retried.
- *Connection timeouts and premature termination:* socket errors, a closed control connection, or `SERVER_TERMINATE` during the test (`_check_ctrl_during_test`) end the run as `premature_close`/`server_terminate`. The samples collected so far are kept; a run whose transfer finished but whose results exchange failed is kept as `incomplete_termination`.
- On any failure after the control connection opens, the client sends `CLIENT_TERMINATE` (`_abort`) so the public server is released at once. Retryable failures are retried up to 2 more times with linear back-off plus jitter.

Validation. For every run, the bytes the server reports having received (its results JSON) match the client's `tcpi_bytes_acked`: median difference 0.21 %, maximum 1.99 % over all runs above 5 MB (`server_rx_MB` vs. `client_acked_MB` in `run_summary.csv`).

2.2 (b) Destination selection

`select_destinations` shuffles the server list (seeded, seed recorded), skips a host already chosen under another port range, and validates each candidate with a 3 s iperf3 test. A failed candidate is logged and replaced, up to $3n$ attempts; any shortfall is printed and stored in `metadata.json`. In this run 15 candidates were tried for $n = 10$: four answered `ACCESS_DENIED` (busy) and one was unreachable, so **all 10 destinations were found (shortfall 0)**.

#	Host	Port	Country	Site	Provider
1	a210.speedtest.wobcom.de	5201	DE	Frankfurt	WOBCOM
2	102.214.66.19	5201	GH	Accra	DATAPACKET
3	122.228.230.122	5202	CN	Ningbo	China Telecom
4	66.35.31.81	30003	JP	Tokyo	FortiSASE
5	iperf.as42831.net	5361	GB	London	UK Servers
6	ping.online.net	5207	FR	Vitry-sur-Seine	Scaleway
7	speedtest.goco.ca	9214	CA	Montreal	TELUS
8	speedtest1.cwpanama.net	5506	PA	Colón	CW Panama
9	49.205.75.2	5010	IN	Bengaluru	ACT Fibernet
10	156.146.52.1	5201	TR	Istanbul	DATAPACKET

Attempt	Candidate	Outcome	Failure kind
1	185.93.1.65:5201	rejected	busy
2	185.102.218.1:5201	rejected	busy
3	a210.speedtest.wobcom.de:5201	selected	–
4	102.214.66.19:5201	selected	–
5	122.228.230.122:5202	selected	–
6	66.35.31.81:30003	selected	–
7	iperf.as42831.net:5361	selected	–
8	138.199.4.1:5201	rejected	busy
9	ping.online.net:5207	selected	–
10	speedtest.goco.ca:9214	selected	–
11	207.211.214.65:5201	rejected	busy
12	speedtest.nl1.mirhosting.net:5207	rejected	connect_error
13	speedtest1.cwpanama.net:5506	selected	–
14	49.205.75.2:5010	selected	–
15	156.146.52.1:5201	selected	–

Table 3: Selected destinations and the full selection log (destinations.csv, selection_log.csv).

2.3 (c) Goodput

At every sample k (every 0.2 s) the client reads TCP_INFO from the data socket and computes (_sample)

$$\text{goodput}_k = \frac{(\texttt{bytes_acked}_k - \texttt{bytes_acked}_{k-1}) \cdot 8}{t_k - t_{k-1}} \text{ [bit/s]},$$

where bytes_acked is the kernel's cumulative count of cumulatively acknowledged bytes (tcpi_bytes_acked) and t_k is a monotonic timestamp taken at the getsockopt() call, so the actual elapsed time is used rather than the nominal interval. Sample 0 is a baseline taken just before the first write.

Figure 1 shows goodput over time for every destination and algorithm (all repetitions overlaid), and Table 4 the distribution over all 0.2 s samples of all repetitions.

Table 4: Goodput per destination and algorithm (Mbit/s) over all 0.2 s samples of all usable repetitions (goodput_summary.csv). The algorithm column is the one read back from the data socket. *Generated by* build_tables.

Destination	Algorithm	Runs	Min	Median	Mean	p95
1 Frankfurt	cubic	2	0.06	2.13	29.97	116.65
	reno	3	0.00	76.08	61.13	111.62
	bbr	2	0.00	2.13	4.54	20.16
2 Accra	cubic	3	0.00	70.95	64.66	101.49
	reno	3	0.00	42.61	41.96	50.66
	bbr	3	0.00	105.65	96.87	117.30

Destination	Algorithm	Runs	Min	Median	Mean	p95
3 Ningbo	cubic	3	0.00	0.17	0.22	0.52
	reno	3	0.00	0.23	0.23	0.52
	bbr	3	0.00	25.02	31.63	90.17
4 Tokyo	cubic	3	0.00	75.05	75.72	122.18
	reno	3	0.00	48.86	47.78	64.41
	bbr	3	0.06	123.82	117.57	151.66
5 London	cubic	3	0.58	102.36	100.74	149.20
	reno	3	0.00	54.58	53.58	85.27
	bbr	3	0.06	152.19	140.58	190.40
6 Vitry-sur-Seine	cubic	3	0.00	110.96	112.41	180.74
	reno	3	0.58	81.52	81.80	129.67
	bbr	3	0.00	151.13	138.04	213.84
7 Montreal	cubic	3	15.57	291.69	295.66	410.87
	reno	3	0.00	226.34	228.30	340.99
	bbr	3	0.93	371.35	366.35	636.32
8 Colón	cubic	3	1.73	127.63	133.14	216.22
	reno	3	0.58	73.18	85.62	173.97
	bbr	3	0.00	192.92	189.62	299.06
9 Bengaluru	cubic	3	0.00	0.23	0.20	0.23
	reno	3	0.00	0.23	0.25	0.23
	bbr	3	0.00	2.09	1.50	2.44
10 Istanbul	cubic	3	0.00	70.61	73.08	124.91
	reno	3	0.00	40.53	36.12	72.57
	bbr	3	0.00	120.12	102.80	162.46

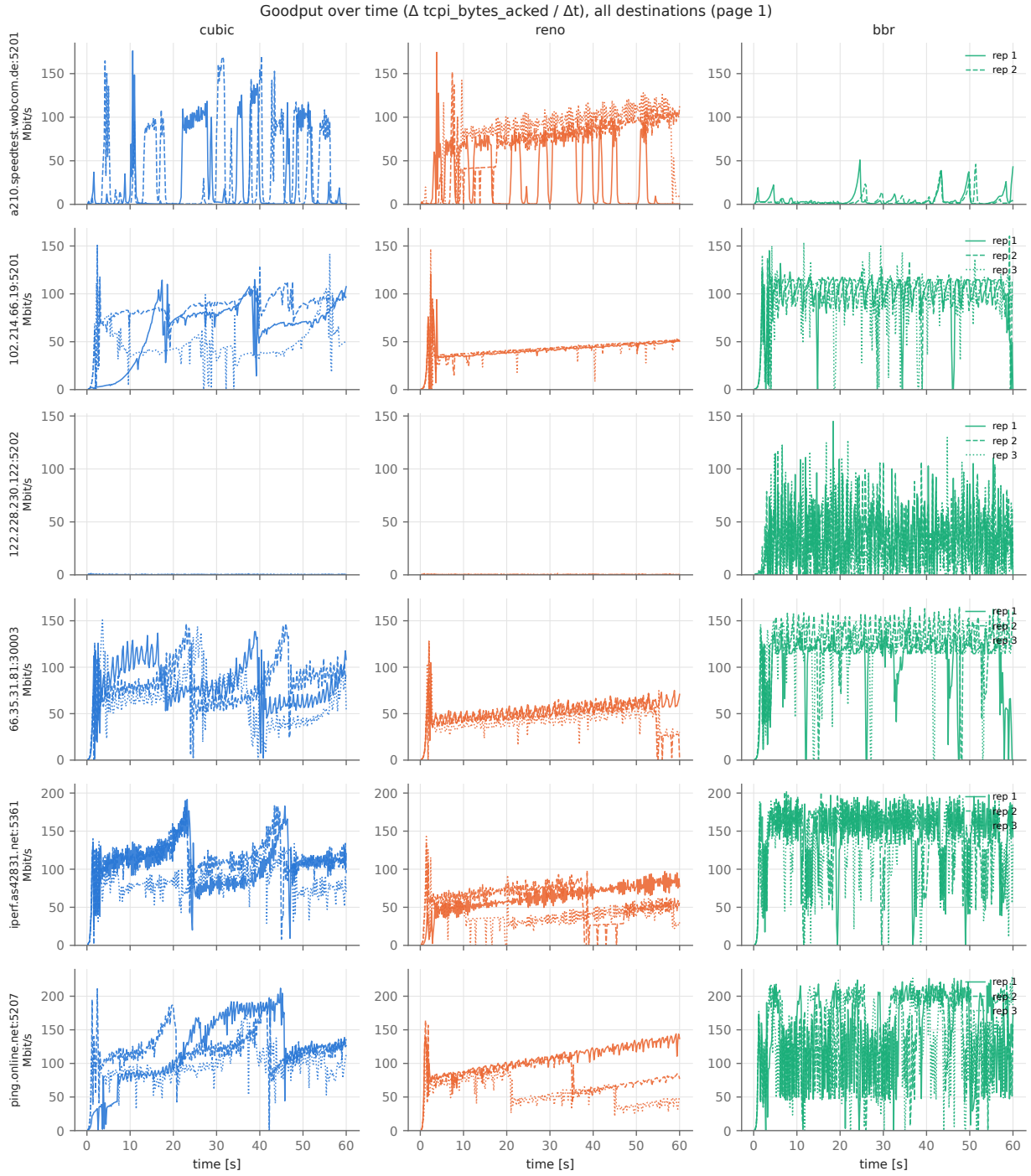


Figure 1: Goodput over time, destinations 1–6 (rows) × algorithm (columns), all repetitions. *Generated by* `plot_goodput_all`.

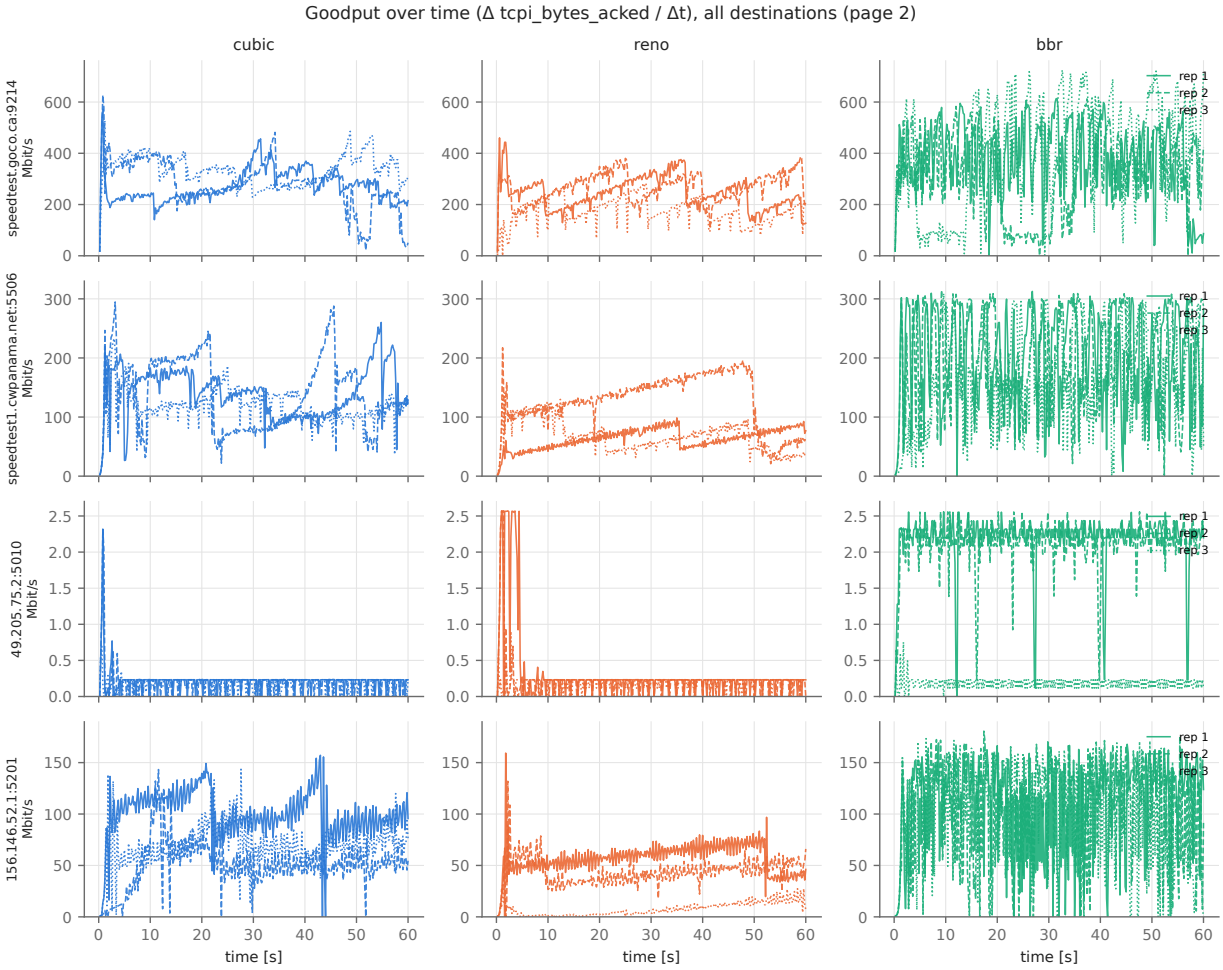


Figure 2: Goodput over time, destinations 7–10. *Generated by* `plot_goodput_all`.

3 Q2: TCP statistics during the transfer

Algorithm. Q2 uses **CUBIC**, the host's default (`net.ipv4.tcp_congestion_control = cubic`). It is set explicitly on the data socket and read back with `getsockopt(TCP_CONGESTION)` after connecting; every CUBIC run reports `cubic`.

3.1 (a) Extracting TCP statistics

Source. `getsockopt(IPPROTO_TCP, TCP_INFO, 512)` on the *data* socket (not the control socket) at each sampling deadline, in the same loop iteration that computes goodput. The kernel copies $\min(512, \texttt{sizeof(struct tcp_info)})$ bytes; this host returned **248 bytes** (logged per sample as `tcp_info_len_B`).

Decoding format. `_FIELDS` lists every field of `struct tcp_info` from `include/uapi/linux/tcp.h` as (name, C type, byte offset), decoded with Python `struct` in native byte order. A field is decoded only if the returned buffer contains it, so older kernels give empty columns instead of garbage. The table actually used on this host (also stored in `metadata.json`):

Field	Type	Off.	Field	Type	Off.	Field	Type	Off.
state	__u8	0	last_ack_recv	__u32	56	data_segs_in	__u32	152
ca_state	__u8	1	pmtu	__u32	60	data_segs_out	__u32	156
retransmits	__u8	2	rcv_ssthresh	__u32	64	delivery_rate	__u64	160
probes	__u8	3	rtt	__u32	68	busy_time	__u64	168
backoff	__u8	4	rttvar	__u32	72	rwnd_limited	__u64	176
options	__u8	5	snd_ssthresh	__u32	76	sndbuf_limited	__u64	184
wscale	__u8	6	snd_cwnd	__u32	80	delivered	__u32	192
app_limited_flags	__u8	7	advmss	__u32	84	delivered_ce	__u32	196
rto	__u32	8	reordering	__u32	88	bytes_sent	__u64	200
ato	__u32	12	rcv_rtt	__u32	92	bytes_retrans	__u64	208
snd_mss	__u32	16	rcv_space	__u32	96	dsack_dups	__u32	216
rcv_mss	__u32	20	total_retrans	__u32	100	reord_seen	__u32	220
unacked	__u32	24	pacing_rate	__u64	104	rcv_ooopack	__u32	224
sacked	__u32	28	max_pacing_rate	__u64	112	snd_wnd	__u32	228
lost	__u32	32	bytes_acked	__u64	120	rcv_wnd	__u32	232
retrans	__u32	36	bytes_received	__u64	128	rehash	__u32	236
fackets	__u32	40	segs_out	__u32	136	total_rto	__u16	240
last_data_sent	__u32	44	segs_in	__u32	140	total_rto_recoveries	__u16	242
last_ack_sent	__u32	48	notsent_bytes	__u32	144	total_rto_time	__u32	244
last_data_recv	__u32	52	min_rtt	__u32	148			

Logged per sample (`samples/<run_id>.csv`, one row per 0.2 s, columns `SAMPLE_COLUMNS`). Each row carries `run_id` (`r<rep>_d<dest>_<alg>`), destination, requested and actual algorithm, repetition, and the unit in the column suffix (`_s`, `_us`, `_B`, `_Bps`, `_seg`):

- **Required:** monotonic timestamp `t_s` (plus wall clock), `snd_cwnd_seg`, RTT estimate `srtt_us`, loss signal `d_retrans_seg`, and `goodput_bps`.
- **Also logged:** `rttvar_us`, `min_rtt_us`, `rto_us`, `snd_ssthresh_seg`, `pacing_rate_Bps`, `delivery_rate_Bps` (the kernel's per-ACK delivery-rate estimate), cumulative `bytes_acked_B`, `bytes_sent_B`, `bytes_retrans_B`, `lost_seg`, `total_rto`, `ca_state` (0 Open, 1 Disorder, 2 CWR, 3 Recovery, 4 Loss), `snd_wnd_B` (receiver's advertised window), and the cumulative `busy_time_us`, `rwnd_limited_us`, `sndbuf_limited_us`.

Interval measurements. Every interval quantity is a difference of successive cumulative counters divided by the *measured* Δt: goodput from `bytes_acked`, `d_retrans_seg` from `tcpi_total_retrans`, `d_bytes_sent_B`, `d_total_rto`.

Loss proxy. TCP cannot observe loss directly, so the loss signal is retransmissions: $\texttt{d_retrans_seg}_k = \texttt{total_retrans}_k - \texttt{total_retrans}_{k-1}$, the number of segments retransmitted in the interval (fast retransmits and RTO retransmits; spurious retransmissions are counted too, so it slightly overestimates loss). Per run we also report the retransmission ratio `bytes_retrans/bytes_sent`, and the RTO count separately.

3.2 (b) Visualization

The representative destination is chosen automatically (`choose_representative`): among the destinations with usable runs for every algorithm, the one whose median goodput is closest to the median over all destinations. Here that is **destination 4, `66.35.31.81:30003` (Tokyo, FortiSASE)**, min RTT ≈ 150–170 ms. The time series show repetition 1, the scatter plots pool all three repetitions.

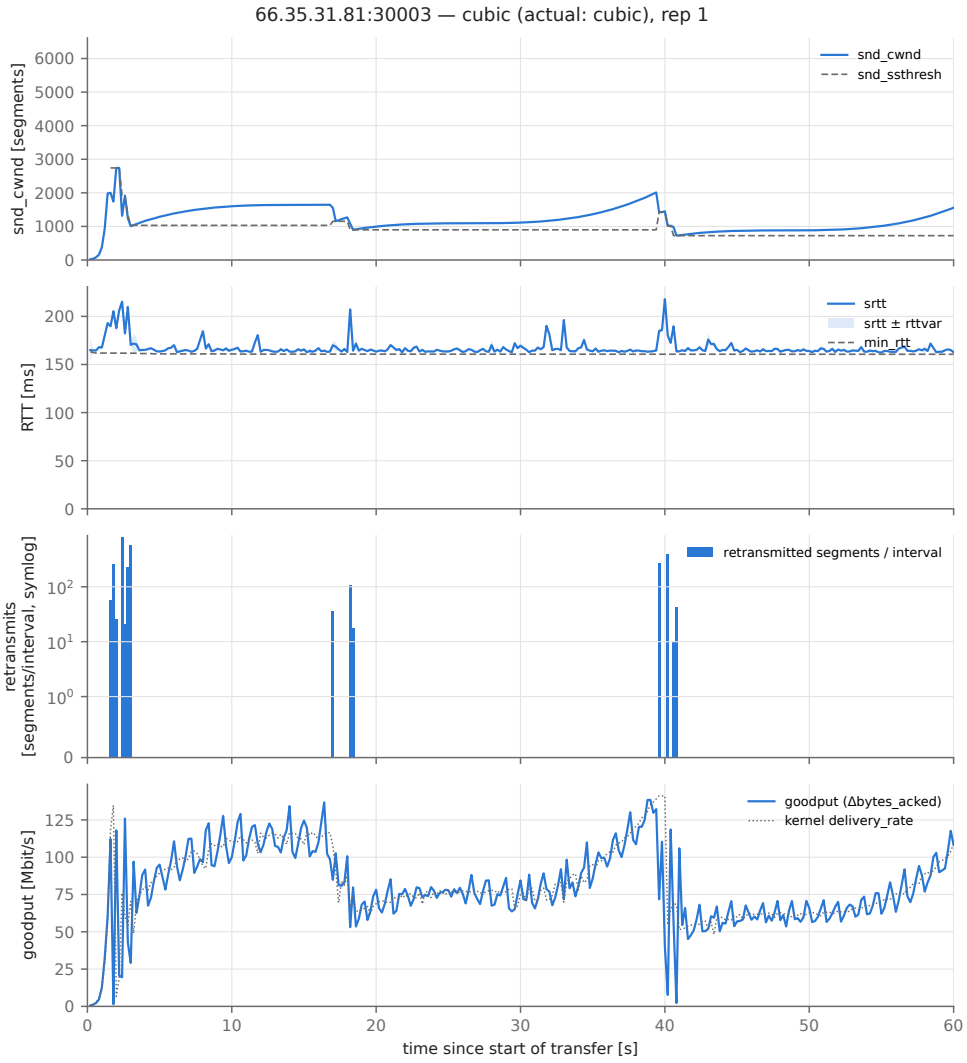


Figure 3: Q2(b)(i) CUBIC to Tokyo, repetition 1: `snd_cwnd` (with `ssthresh`), smoothed RTT (± rttvar, with min RTT), retransmitted segments per 0.2 s interval (loss proxy, symlog), and goodput (with the kernel's delivery-rate estimate). *Generated by* `plot_timeseries`.

3.3 (c) Observations

What each metric means and how it should affect goodput in congestion avoidance.

- **snd_cwnd** is the most data the sender may have unacknowledged. A window-limited sender delivers about one window per round trip, so goodput ≈ cwnd · MSS/RTT as long as cwnd is below the bandwidth-delay product (BDP); above it the link is full and extra window only adds queue. This is

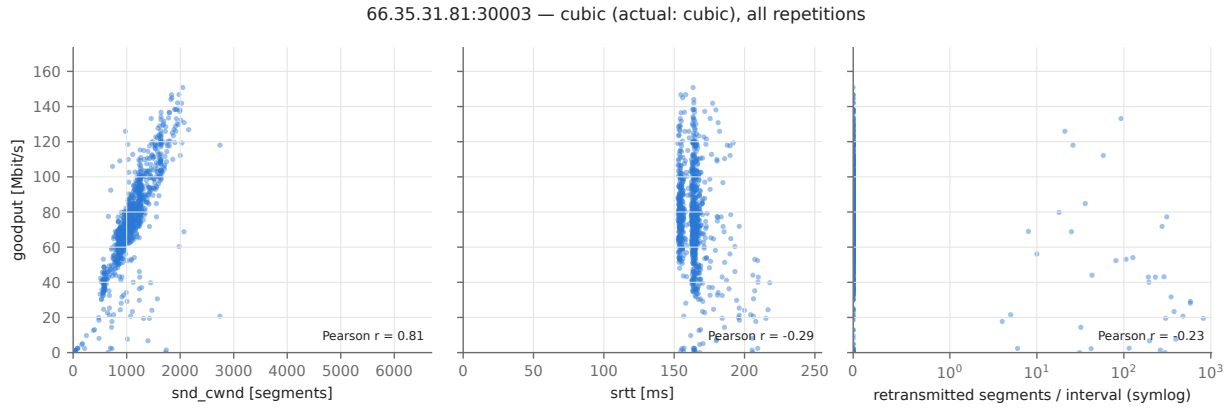


Figure 4: Q2(b)(ii) CUBIC to Tokyo, all repetitions, one point per 0.2 s sample: `snd_cwnd`, sRTT and retransmissions vs. goodput, with Pearson r. *Generated by* `plot_scatter`.

the strongest relationship in the data: Pearson $r = 0.81$ for CUBIC in Figure 4. For example, at 15 s cwnd is 1,641 segments and sRTT 165.8 ms, which predicts $1641 \cdot 1448 \cdot 8/0.1658 = 115$ Mbit/s; the measured goodput is 124 Mbit/s and the delivery rate 109 Mbit/s.

- **sRTT** is propagation delay (its floor, min RTT, 160.6 ms here) plus queueing. In congestion avoidance a rising RTT at constant goodput means the window exceeds the BDP and a queue is building; a large RTT also lowers goodput for a given cwnd. Here RTT mostly stays within 10 ms of the minimum, and the correlation with goodput is weak and negative ($r = -0.29$): the RTT spikes (up to 218 ms at 40 s) coincide with the losses and the goodput drops after them.
- **Loss proxy (retransmitted segments per interval).** Each loss episode makes CUBIC multiply cwnd by $\beta = 0.7$ and enter recovery, so goodput falls right after a burst of retransmissions; an RTO would reset cwnd to 1. The correlation is negative ($r = -0.23$) and the effect is clearly visible in the time series, but most intervals have no retransmissions at all.
- **Goodput** is the outcome: bytes newly acknowledged per interval, capped by cwnd/RTT, the receiver's window, and the bottleneck.

What the CUBIC trace shows (Figure 3):

1. *Slow-start overshoot.* cwnd doubles to 2,742 segments by 2.0 s, beyond the path's BDP (≈ 1,950 segments). Between 1.6 and 3.0 s, 1,999 segments are retransmitted in seven intervals, and cwnd/ssthresh settle at about 1,030.
2. *CUBIC's window curve.* After that, cwnd grows concavely (1,287 at 5 s, 1,602 at 10 s, 1,641 at 15 s) and flattens as it approaches the window of the last loss, $W_{\max}$. Two loss episodes at 17.0 and 18.2 s cut it to about 940 segments; it then stays almost flat around 1,060–1,110 from 22 to 30 s (the plateau near the new $W_{\max}$) and grows convexly to 1,745 by 38 s as CUBIC probes beyond it. The plateau length matches CUBIC's $K = \sqrt[3]{W_{\max}(1-\beta)/C}$: with $W_{\max} \approx 1{,}280$, $\beta = 0.7$, $C = 0.4$, $K \approx 9.9$ s after the 18.2 s loss. Note that CUBIC's growth depends on wall-clock time, not on the number of RTTs.
3. *Goodput follows cwnd*: ≈ 95–125 Mbit/s from 5 to 17 s, 68–85 Mbit/s from 19 to 30 s, about 110 Mbit/s at 38 s, and down to 40–66 Mbit/s after the third loss episode at 39.6–40.8 s, climbing back to 118 Mbit/s by 60 s.
4. *Losses with almost no queue.* sRTT averages only 6.5 ms above min RTT and spikes (to 218 ms at 40 s) only around the loss episodes. The bottleneck therefore drops packets with very little standing queue: either a shallow buffer or competing traffic; this is the regime in which loss-based algorithms lose throughput (Section 4).

Anomalous behaviour in the traces.

- **Receiver-limited server (destination 1, Frankfurt).** Goodput switches between ≈ 100 Mbit/s and near zero for all algorithms (Figure 1, top row) while cwnd stays above 1,000 segments. The cause is the server: its advertised window (`snd_wnd_B`) repeatedly collapses to 8 KB and reopens to 13–24 MB, and the kernel reports the connection receive-window-limited for 45.6 of 60 s in CUBIC repetition 1 (`rwnd_limited_us`). The iperf3 server is not draining its socket, so this destination measures the server, not the network or the CC algorithm.
- **No window scaling (destination 9, Bengaluru).** The largest advertised window in any run is 65,152 bytes, i.e. the server does not use TCP window scaling. That caps throughput at $65152 \cdot 8/0.224\,\text{s} = 2.3$ Mbit/s, exactly where BBR flat-lines (2.2–2.3 Mbit/s in repetitions 1 and 2, zero retransmissions). In BBR repetition 3 the window dropped to 5,888 bytes, giving 0.2 Mbit/s.
- **Very lossy path (destination 3, Ningbo).** CUBIC and Reno never exceed ≈ 0.5 Mbit/s: retransmissions are 4–11 % of sent bytes, cwnd stays at 2–7 segments with `ssthresh` at 2, and there are up to two RTOs per run. The rate is what the loss-based throughput model predicts: $1.22 \cdot \text{MSS}/(\text{RTT}\sqrt{p}) = 1.22 \cdot 11584\,\text{bit}/(0.22\,\text{s} \cdot \sqrt{0.05}) \approx 0.29$ Mbit/s. BBR on the same path gets 26–35 Mbit/s (Section 4).

4 Q3: Comparing CUBIC, Reno and BBR, and a new algorithm

4.1 (a) Experimental design

All three algorithms use the same 10 destinations, 60 s transfers, a 0.2 s sampling interval, and **3 repetitions**: 90 runs. The algorithm is set with `setsockopt(TCP_CONGESTION)` before `connect()` and read back afterwards; every usable run used the requested algorithm. Within a repetition every destination is tested with all three algorithms back to back, and the order rotates by (repetition + destination index), so no algorithm is always first at a destination (`run_experiments`). **88 of 90 runs are usable**: five runs needed 2–3 attempts, and the Frankfurt server stayed busy for repetition 3 of CUBIC and BBR (3 attempts each), so those two runs are missing.

Table 5: Test order (`test_order.csv`), grouped by repetition and destination. Seq is the global position of the three runs.

Rep	Destination	Seq	Algorithm order	Status
1	1 Frankfurt	1–3	cubic, reno, bbr	ok, ok, ok
1	2 Accra	4–6	reno, bbr, cubic	ok, ok, ok
1	3 Ningbo	7–9	bbr, cubic, reno	ok, ok, ok
1	4 Tokyo	10–12	cubic, reno, bbr	ok, ok, ok
1	5 London	13–15	reno, bbr, cubic	ok, ok (2 tries), ok
1	6 Vitry-sur-Seine	16–18	bbr, cubic, reno	ok, ok, ok
1	7 Montreal	19–21	cubic, reno, bbr	ok, ok, ok
1	8 Colón	22–24	reno, bbr, cubic	ok, ok, ok
1	9 Bengaluru	25–27	bbr, cubic, reno	ok, ok, ok
1	10 Istanbul	28–30	cubic, reno, bbr	ok, ok, ok
2	1 Frankfurt	31–33	reno, bbr, cubic	ok (3 tries), ok, ok
2	2 Accra	34–36	bbr, cubic, reno	ok, ok, ok
2	3 Ningbo	37–39	cubic, reno, bbr	ok, ok, ok
2	4 Tokyo	40–42	reno, bbr, cubic	ok, ok, ok
2	5 London	43–45	bbr, cubic, reno	ok, ok, ok
2	6 Vitry-sur-Seine	46–48	cubic, reno, bbr	ok (2 tries), ok, ok
2	7 Montreal	49–51	reno, bbr, cubic	ok, ok, ok
2	8 Colón	52–54	bbr, cubic, reno	ok, ok, ok
2	9 Bengaluru	55–57	cubic, reno, bbr	ok, ok, ok
2	10 Istanbul	58–60	reno, bbr, cubic	ok, ok (2 tries), ok
3	1 Frankfurt	61–63	bbr, cubic, reno	failed (3 tries), failed (3 tries), ok (2 tries)
3	2 Accra	64–66	cubic, reno, bbr	ok, ok, ok
3	3 Ningbo	67–69	reno, bbr, cubic	ok, ok, ok
3	4 Tokyo	70–72	bbr, cubic, reno	ok, ok, ok
3	5 London	73–75	cubic, reno, bbr	ok, ok, ok
3	6 Vitry-sur-Seine	76–78	reno, bbr, cubic	ok, ok, ok
3	7 Montreal	79–81	bbr, cubic, reno	ok, ok, ok
3	8 Colón	82–84	cubic, reno, bbr	ok, ok, ok
3	9 Bengaluru	85–87	reno, bbr, cubic	ok, ok, ok
3	10 Istanbul	88–90	bbr, cubic, reno	ok, ok, ok

4.2 (a) Results

Figures 5–12 repeat the Q2(b) plots for each algorithm at the same representative destination with identical axes, then show the algorithms side by side and overlaid, and the per-destination comparison. Tables 6 and 7 summarise all destinations.

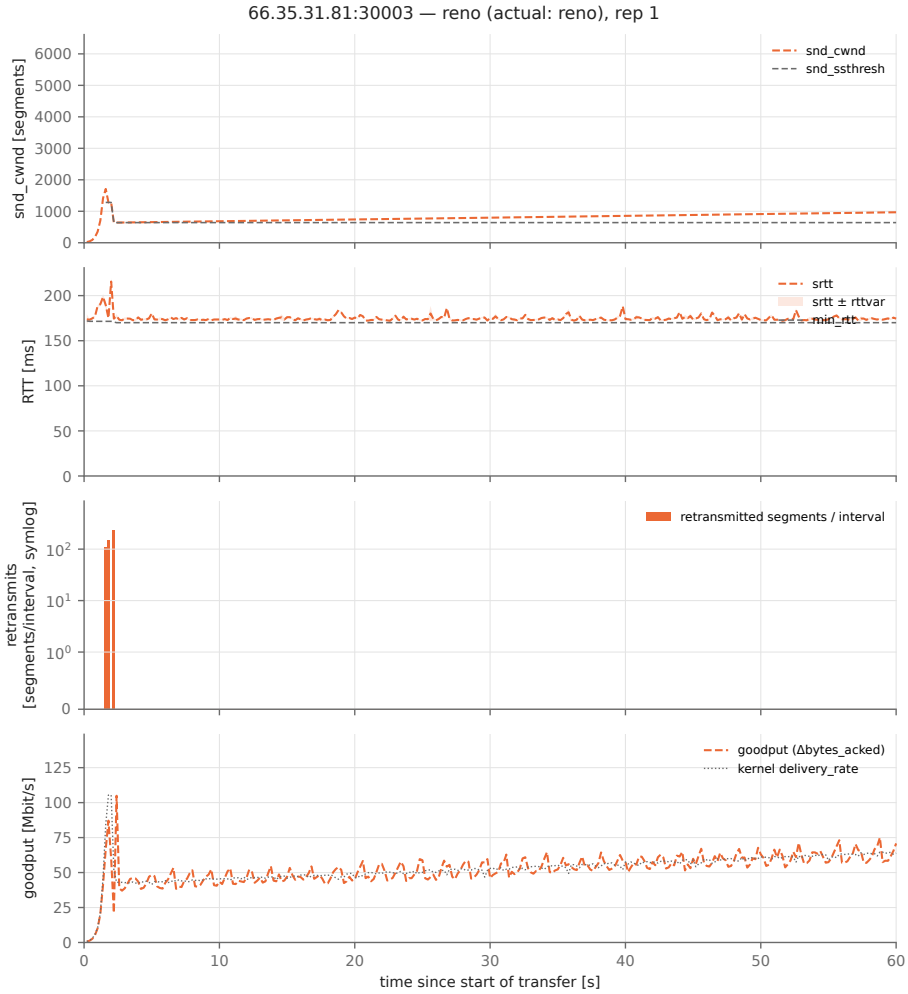


Figure 5: Reno to Tokyo, repetition 1 (same axes as Figure 3). *Generated by* `plot_timeseries`.

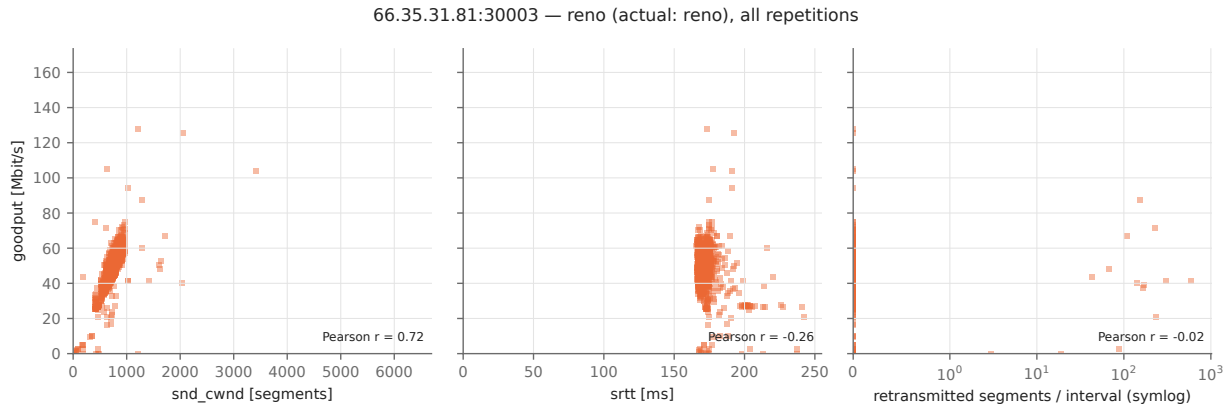


Figure 6: Reno to Tokyo, all repetitions: cwnd, sRTT and retransmissions vs. goodput. *Generated by* `plot_scatter`.

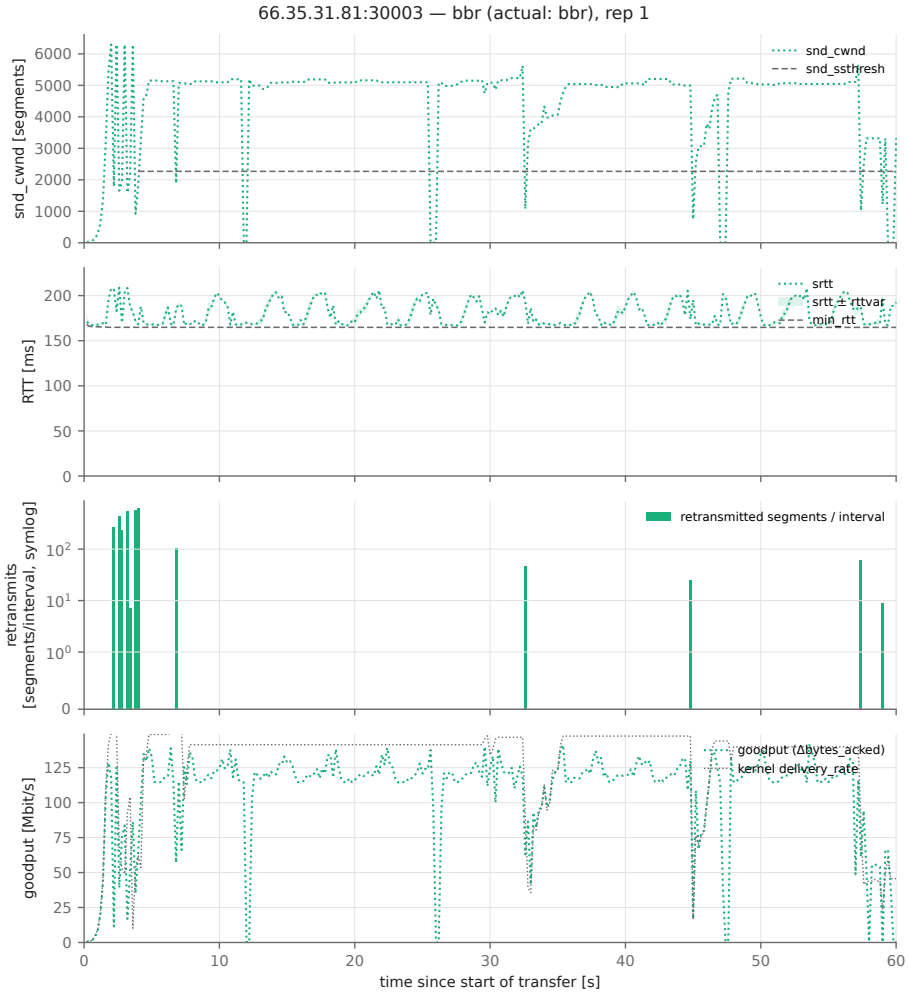


Figure 7: BBR to Tokyo, repetition 1 (same axes). *Generated by* `plot_timeseries`.

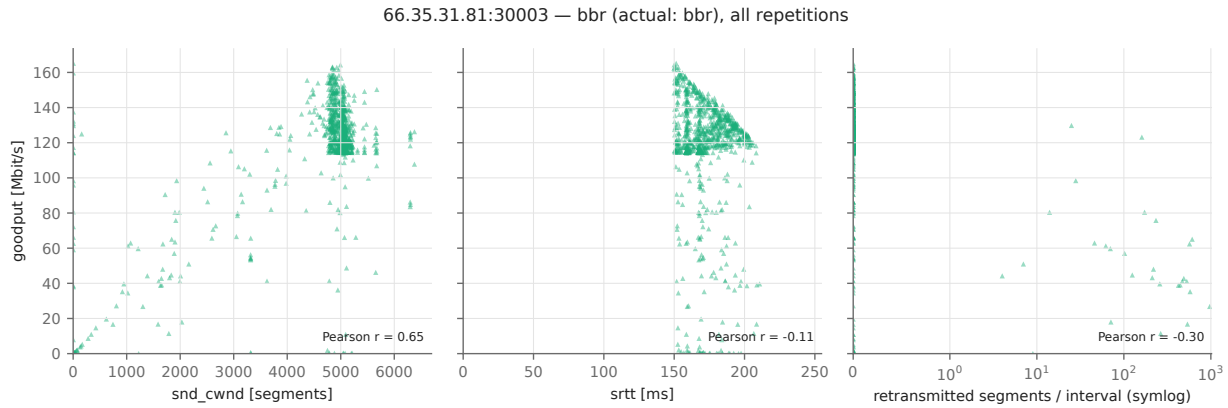


Figure 8: BBR to Tokyo, all repetitions. *Generated by* `plot_scatter`.

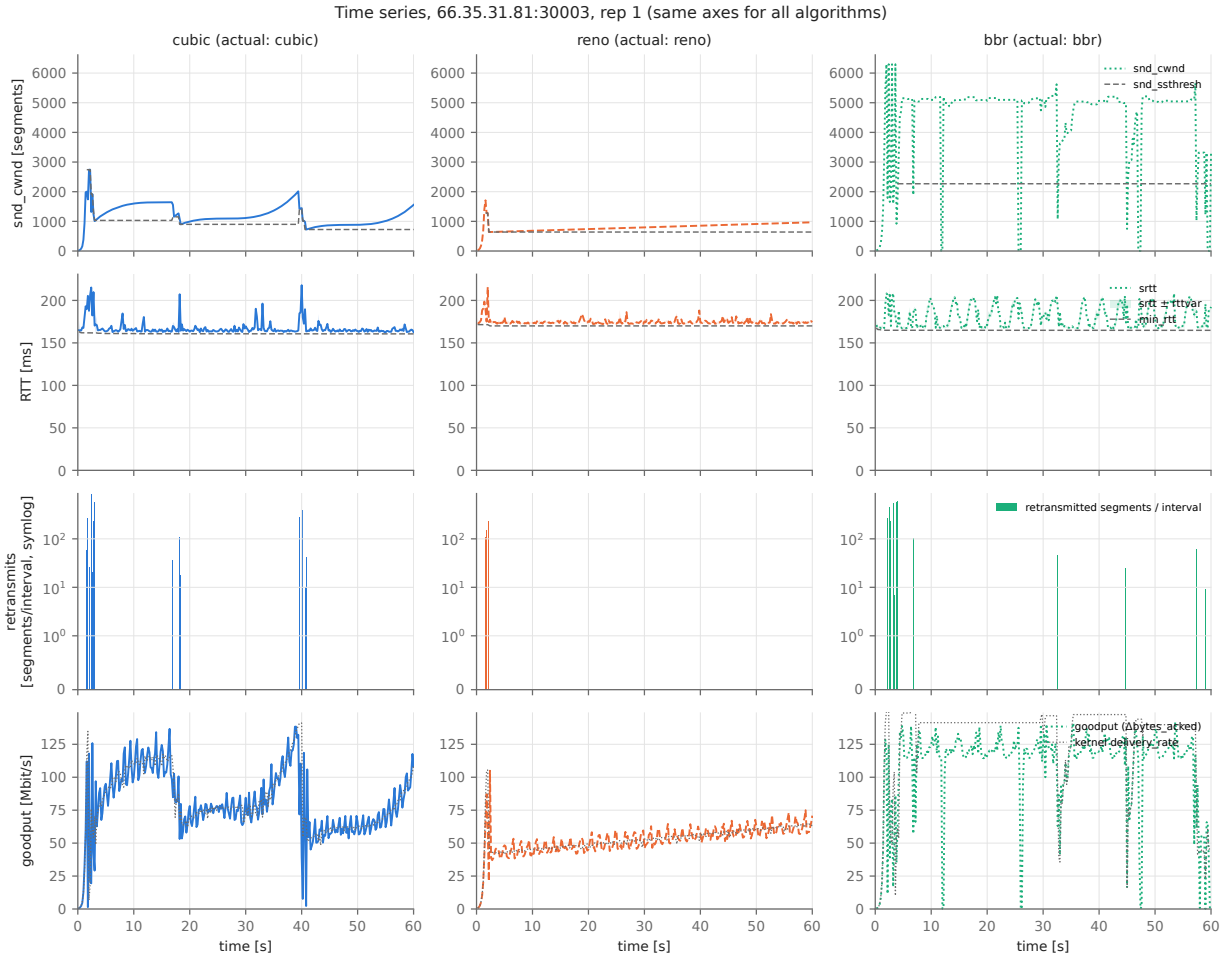


Figure 9: CUBIC, Reno and BBR to Tokyo side by side, repetition 1, shared axes. *Generated by* `make_plots`.

Table 6: Mean goodput (Mbit/s), mean ± standard deviation over repetitions of the per-run means; best per destination in bold. Runs = usable runs for CUBIC/Reno/BBR. (`algorithm_comparison.csv`)

Destination	Runs	CUBIC	Reno	BBR	BBR/CUBIC
1 Frankfurt	2/3/2	30.0 ± 2.1	**61.1** ± 19.7	4.5 ± 0.6	0.15
2 Accra	3/3/3	64.7 ± 15.5	42.0 ± 0.2	**96.9** ± 1.1	1.50
3 Ningbo	3/3/3	0.2 ± 0.1	0.2 ± 0.0	**31.6** ± 4.0	145.63
4 Tokyo	3/3/3	75.7 ± 7.3	47.8 ± 4.0	**117.6** ± 7.4	1.55
5 London	3/3/3	100.7 ± 12.6	53.6 ± 10.0	**140.6** ± 5.3	1.40
6 Vitry-sur-Seine	3/3/3	112.4 ± 11.3	81.8 ± 20.5	**138.0** ± 7.1	1.23
7 Montreal	3/3/3	295.7 ± 27.3	228.3 ± 39.3	**366.4** ± 34.5	1.24
8 Colón	3/3/3	133.1 ± 9.6	85.6 ± 28.9	**189.6** ± 16.2	1.42
9 Bengaluru	3/3/3	0.2 ± 0.0	0.2 ± 0.0	**1.5** ± 0.9	7.36
10 Istanbul	3/3/3	73.1 ± 20.5	36.1 ± 20.9	**102.8** ± 4.4	1.41
All	29/30/29	90.6 ± 82.8	63.7 ± 64.3	**122.9** ± 101.4	1.36

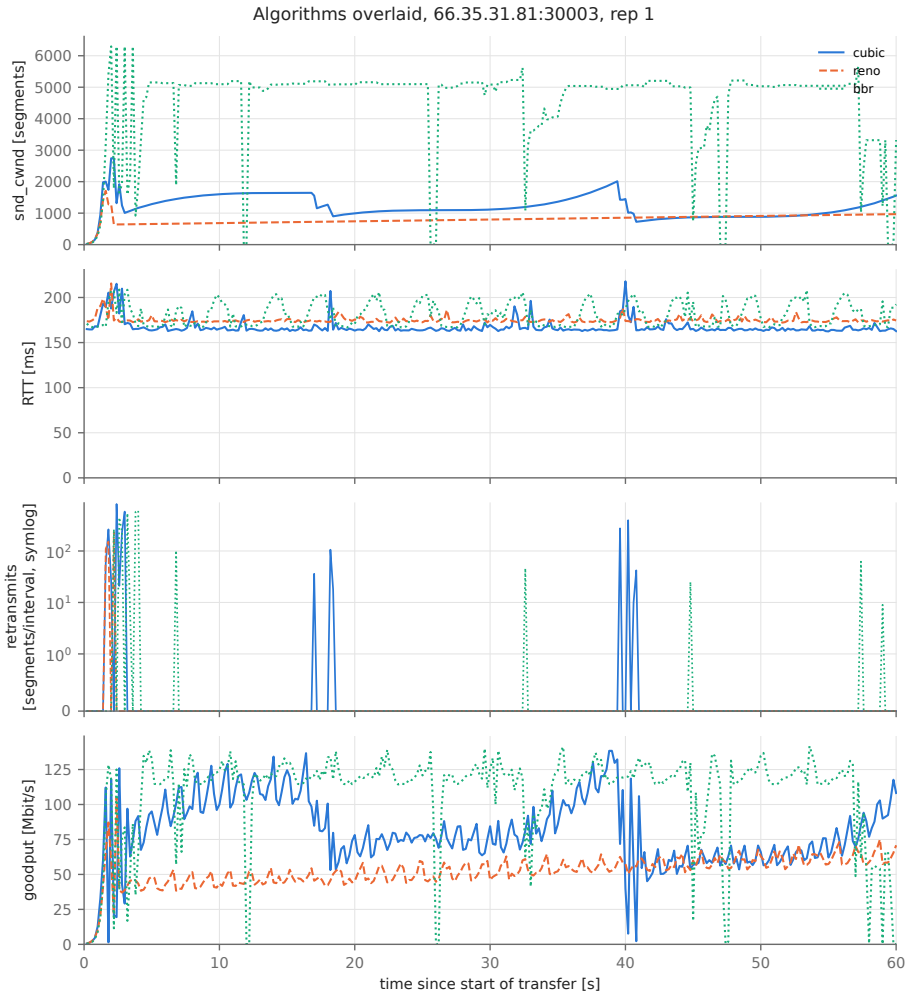


Figure 10: The same three runs overlaid on shared panels. *Generated by* `make_plots`.

Table 7: Mean smoothed RTT (ms, mean ± std over repetitions; lowest in bold) and retransmission ratio (% of bytes sent).

	sRTT [ms]			Retransmitted [%]		
Destination	CUBIC	Reno	BBR	CUBIC	Reno	BBR
1 Frankfurt	**127.5** ± 0.2	128.6 ± 4.8	130.0 ± 4.4	1.87	0.49	0.00
2 Accra	207.4 ± 1.5	**200.8** ± 3.4	209.5 ± 2.0	0.52	0.55	0.55
3 Ningbo	222.3 ± 4.4	218.9 ± 1.9	**213.5** ± 12.6	8.22	7.04	8.92
4 Tokyo	**163.9** ± 5.2	173.1 ± 1.2	172.7 ± 6.3	0.59	0.32	0.42
5 London	**129.8** ± 1.8	136.4 ± 4.9	138.4 ± 2.1	0.66	0.43	0.85
6 Vitry-sur-Seine	116.2 ± 2.4	**107.5** ± 3.6	122.7 ± 1.1	0.38	0.35	4.09
7 Montreal	36.3 ± 0.9	**35.4** ± 2.3	45.0 ± 2.0	0.13	0.15	6.01
8 Colón	81.9 ± 1.1	**79.6** ± 1.8	87.5 ± 3.2	0.66	0.18	5.53
9 Bengaluru	234.8 ± 2.7	**234.8** ± 2.1	237.0 ± 13.1	5.88	4.77	1.41
10 Istanbul	154.5 ± 2.4	**152.7** ± 5.3	165.2 ± 1.1	0.39	0.38	2.39
All	148.1 ± 60.5	**146.8** ± 59.7	152.9 ± 57.8	1.93	1.47	3.12

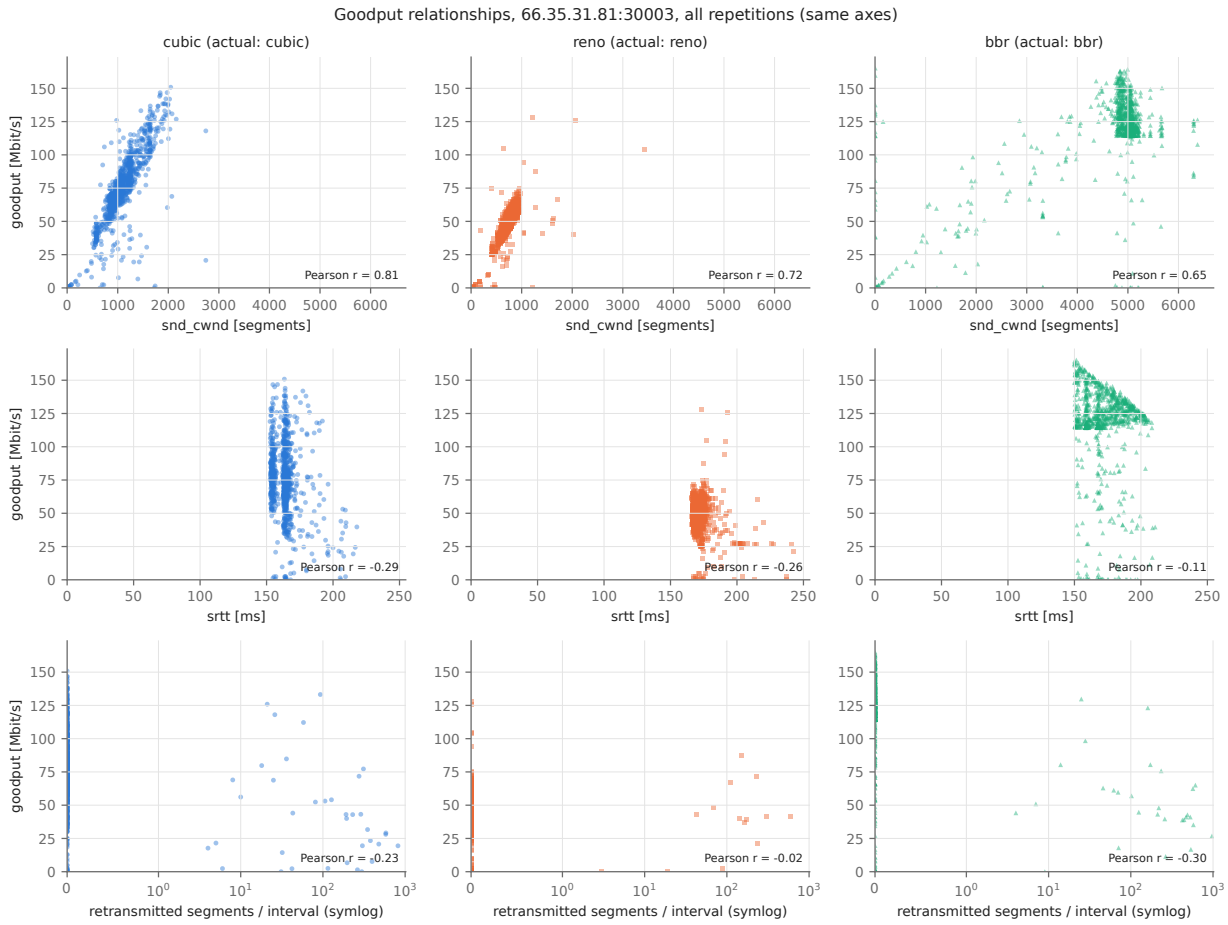


Figure 11: Scatter grid, metric (rows) × algorithm (columns), Tokyo, all repetitions, shared axes. *Generated by* `make_plots`.

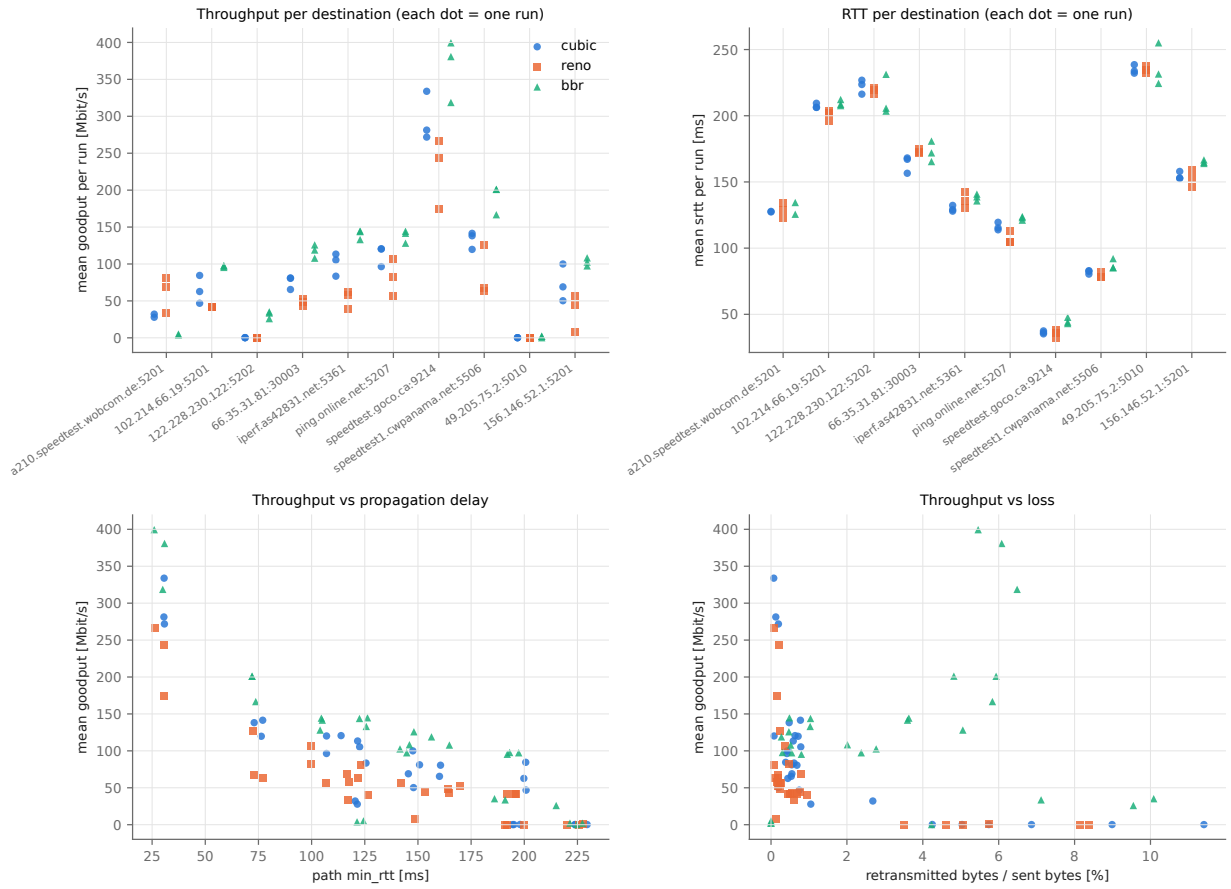


Figure 12: All destinations, one point per run: mean goodput and mean sRTT per destination, goodput vs. path min RTT, and goodput vs. retransmission ratio. *Generated by* `plot_regimes`.

4.3 (a) Operating regimes

Taking the per-destination results together, the paths fall into four regimes. For the summary numbers below, the three paths whose behaviour is dominated by the server or by extreme loss (Frankfurt, Bengaluru, Ningbo) are excluded; over the other seven destinations (21 runs per algorithm):

	Mean goodput	RTT inflation	Median cwnd/BDP	Retransmitted	Mean CoV over reps
CUBIC	122.2 Mbit/s	6.7 ms	0.57	0.47 %	0.14
Reno	82.2 Mbit/s	7.1 ms	0.51	0.34 %	0.23
BBR	164.5 Mbit/s	16.5 ms	1.66	2.83 %	0.06

RTT inflation is mean sRTT minus min RTT; BDP is estimated per run as max(delivery rate) × min RTT (`run_stats`); CoV is the standard deviation of the per-run mean goodput divided by its mean, per destination.

1. Long-RTT, high-BDP paths with light loss (Accra, Tokyo, London, Paris, Panama, Istanbul; min RTT 72–200 ms, BDP ≈ 2.8–3.5 MB). *Throughput: BBR > CUBIC > Reno* at every one of them (BBR 1.23–1.55× CUBIC, CUBIC 1.37–2.0× Reno). *RTT: Reno or CUBIC lowest everywhere, BBR highest at five of the six.*

- The loss-based algorithms never fill the pipe: their mean cwnd is only about 0.5–0.6 of the BDP. Losses on these paths come in a few short episodes (in the Tokyo CUBIC run, 14 intervals in 60 s) with almost no RTT increase beforehand, so the bottleneck drops packets before a queue forms (shallow buffers or cross traffic). Each episode cuts the window (CUBIC to 0.7×, Reno to 0.5×), and the window then has to be rebuilt.
- How fast it is rebuilt separates CUBIC from Reno. Reno adds one segment per RTT: at Tokyo it grows from 643 to 967 segments in 56.8 s, exactly one segment per 175 ms RTT, and would need about 230 s to reach the ≈ 1,950-segment BDP. CUBIC's growth depends on elapsed time, not RTT count, so it returns to $W_{\max}$ within about 10 s regardless of RTT. That is why CUBIC's advantage over Reno grows with RTT: 1.30× at Montreal (30 ms) versus 1.5–2.0× at 120–200 ms.
- BBR does not use loss as its congestion signal. It paces at its estimate of the bottleneck bandwidth and caps inflight at twice the estimated BDP (measured median cwnd/BDP 1.66). Isolated losses therefore barely change its rate: Tokyo goodput holds at 120–135 Mbit/s apart from the ProbeRTT dips.
- The price is queueing and loss. BBR adds 13–22 ms of RTT (mean 16.5 ms, vs. mostly 4–10 ms for CUBIC/Reno), with the periodic RTT oscillation of its 1.25× bandwidth probes visible in the time series, and it retransmits 0.3–6 % of its bytes because it keeps sending at full rate into a full buffer. Every ≥ 10 s it drains to cwnd = 4 for ProbeRTT (at 12, 26, 47 and 59 s in the Tokyo run), which shows up as goodput dips to near zero.

2. Short-RTT, high-capacity path (Montreal, 30 ms, 175–400 Mbit/s). The ranking is the same, but the gaps are smaller (BBR 1.24× CUBIC, CUBIC 1.30× Reno): with a 30 ms RTT even Reno's one-segment-per-RTT growth recovers quickly. BBR's costs are largest here relative to the path: sRTT 45 ms vs. 35–36 ms for Reno and CUBIC (+25 %), and 6.0 % of bytes retransmitted vs. 0.13–0.15 %. If latency or friendliness to other flows matters, CUBIC is the better choice on such a path.

3. High random loss (Ningbo, 4–11 % of bytes retransmitted). The loss-based algorithms collapse to 0.2 Mbit/s (cwnd 2–7 segments, consistent with $1/\sqrt{p}$ scaling), while BBR achieves 31.6 Mbit/s, **146× CUBIC**, with even slightly lower RTT. This is the regime BBR was designed for: when loss is not caused by congestion, reacting to it is pure loss of throughput. BBR is in loss recovery in 97–99 % of its samples and retransmits ≈ 9 % of its bytes, which would be very unfriendly to other flows on a congested link.

4. Receiver- or server-limited paths (Frankfurt, Bengaluru). The algorithm is not the bottleneck, so the ranking is meaningless. At Bengaluru BBR reaches the 64 KB/RTT window ceiling (2.2 Mbit/s) while CUBIC and Reno additionally lose packets and suffer 2–4 RTOs per run (0.2 Mbit/s). At Frankfurt, BBR is worst (4.5 Mbit/s): its bandwidth estimate is built from delivery-rate samples, and while the receiver

throttles delivery those samples are low; the estimate likely decays to the throttled rate, and BBR grows its rate only by 25 % per probe cycle, too slowly to use the short periods when the window reopens. CUBIC and Reno keep their large cwnd while receiver-limited and send a full window whenever it reopens, hence the 100 Mbit/s bursts.

Summary. BBR gives the highest throughput wherever the path is long, high-BDP, or randomly lossy, i.e. wherever losses are not a reliable sign of a full queue, at the cost of 10–20 ms more RTT and much more retransmission. CUBIC is the best loss-based compromise: about 0.75× BBR's throughput with less than half its RTT inflation and a sixth of its retransmissions. Reno has the lowest throughput at all seven destinations of regimes 1 and 2, because its linear growth cannot rebuild a window of hundreds of segments within the test. For the lowest RTT, Reno or CUBIC wins at 9 of 10 destinations.

Variability across runs. BBR is also the most consistent (mean CoV 0.06), because its rate follows a bandwidth estimate rather than the timing of individual losses. Reno is the least consistent (0.23): its result depends on when the first loss happens. In Istanbul repetition 3, early losses left `ssthresh` at only 6–11 segments, after which linear growth only reached 219 segments by 60 s: 7.5 Mbit/s, versus 44–57 Mbit/s in the other two repetitions. CUBIC's spread comes from the number of loss episodes per run (Istanbul: 50–100 Mbit/s).

Limitations of comparing over the public Internet.

- The three algorithms never ran at the same time: runs of the same destination are 1–3 minutes apart and repetitions about 30 minutes apart, so cross traffic, server load and even routes change between them. Min RTT to Tokyo varied from 148 to 170 ms across runs.
- Three repetitions give rough standard deviations only; some per-destination differences (e.g. CUBIC vs. Reno at Istanbul, 73 ± 21 vs. 36 ± 21 Mbit/s) are within run-to-run noise.
- Some destinations are limited by the server (receive window, missing window scaling, busy servers) rather than the path, and public servers may shape or police test traffic.
- All tests ran from a single host on one network, one flow at a time, so the results say nothing about fairness between competing flows. The BDP estimate relies on the kernel's delivery-rate samples.
- The host's `bbr` is Linux's BBR version 1; newer BBR versions react to loss and would retransmit less.

4.4 (b) Designed algorithm: Gentle Reno

Pseudocode (custom_tcp.txt; executable version used in the evaluation: GentleReno):

```
# Custom TCP Algorithm

General idea: gentler Reno

Parameters:  β = 0.7            multiplicative-decrease factor
             a = 0.5            additive increase, in packets per RTT
             D = max(4 ms, min_rtt/8)    RTT-rise threshold for leaving slow start
State:       cwnd = 10, ssthresh = ∞, min_rtt = ∞, full_bw = 0, flat_rounds = 0

On every ACK:
    min_rtt ← min(min_rtt, latest RTT sample)
    if in slow start (cwnd < ssthresh):
        cwnd ← cwnd + 1                          # doubles each RTT
    else:
        cwnd ← cwnd + a / cwnd                   # +a packets per RTT

Once per RTT, during slow start only:
    bw ← bytes delivered in the last RTT / RTT       # measured throughput
    if bw ≥ 1.25 · full_bw: full_bw ← bw; flat_rounds ← 0
    else:                   flat_rounds ← flat_rounds + 1
    if (srtt - min_rtt > D) or (flat_rounds ≥ 3):   # queue is building or the pipe is full
        ssthresh ← cwnd                          # leave slow start without cutting

On 3 duplicate ACKs / SACK loss (fast retransmit):
    ssthresh ← max(β · cwnd, 2)
    cwnd ← ssthresh                              # at most once per RTT (one cut per loss event)
    fast recovery as in Reno / NewReno

On retransmission timeout (RTO):
    ssthresh ← max(β · cwnd, 2)
    cwnd ← 1                                     # severe congestion: restart with slow start
    RTO ← 2 · RTO (exponential backoff); RTO is computed from srtt and rttvar
```

Design choices, from the measurements.

- **Gentler decrease, $\beta = 0.7$ (Reno: 0.5).** On the clean long paths, the loss-based algorithms spent the test at only 0.5–0.6 of the BDP because a few short loss episodes cut their window, and Reno was slowest to recover. With $\beta = 0.7$, a single flow keeps the link busy through a loss whenever the buffer holds at least $(1-\beta)/\beta \approx 0.43$ BDP; Reno needs a full BDP of buffer.
- **Additive increase $a = 0.5$ packet/RTT, chosen for fairness with Reno.** The AIMD throughput at loss rate p scales as $\sqrt{a(1+\beta)/(2(1-\beta)p)}$. With $a = 3(1-\beta)/(1+\beta) \approx 0.53$, a Gentle Reno flow gets the same rate as Reno at the same p, so the gentler decrease does not take bandwidth from Reno flows.
- **Slow-start exit on RTT rise or throughput plateau.** The worst loss bursts in our traces came from slow start: CUBIC to Tokyo overshot to 2,742 segments ($\approx 1.4\times$ BDP) and retransmitted about 2,000 segments, and BBR's startup retransmitted about 2,700. Gentle Reno leaves slow start, without cutting cwnd, as soon as $\text{sRTT} - \text{min_RTT} > \max(4\,\text{ms}, \text{min_RTT}/8)$ (a queue is forming) or the measured delivery rate grew by less than 25 % for three rounds (the pipe is full). This uses the throughput signal the way BBR's startup does, and the RTT signal the way CUBIC's HyStart does.
- **Loss stays the congestion signal in steady state.** BBR's higher throughput came with 10–20 ms more queueing and up to 6–10 % retransmissions. Gentle Reno keeps loss-based AIMD (one cut per loss event, RTO resets cwnd to 1 with exponential back-off) so that it backs off on congested links and coexists with Reno and CUBIC.

Evaluation. The host kernel only lets the iperf3 client select built-in algorithms, so Gentle Reno was evaluated against Reno in a fluid model of one FIFO drop-tail bottleneck (50 Mbit/s, 40 ms RTT; `simulate_bottleneck`). Each flow sends cwnd/(base RTT + queueing delay), and arrivals beyond the buffer are dropped in proportion to each flow's rate.

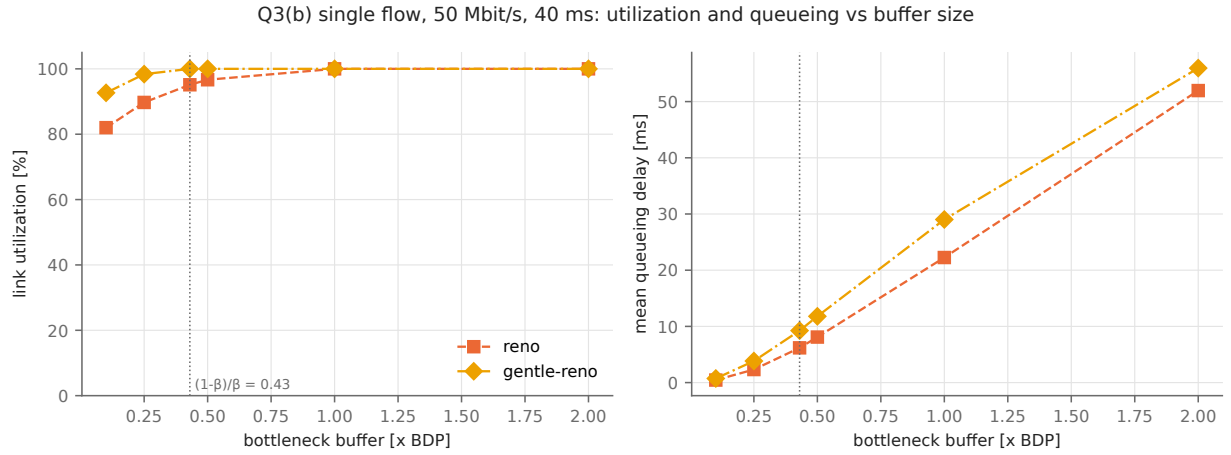


Figure 13: Single flow: link utilization and mean queueing delay vs. bottleneck buffer size (in BDP); the dotted line marks $(1-\beta)/\beta = 0.43$. *Generated by* `make_design_outputs`.

Table 8: Simulation results, 50 Mbit/s bottleneck, 40 ms RTT; split = throughput of the two flows in Mbit/s (`q3b_simulation.csv`).

Scenario	Algorithm	Util. %	Queue [ms]	Loss %	Lost pkts	Split	Jain
1 flow, buffer 0.10 x BDP	reno	82.0	0.5	0.07	–	–	–
1 flow, buffer 0.25 x BDP	reno	89.7	2.3	0.04	–	–	–
1 flow, buffer 0.43 x BDP	reno	95.1	6.2	0.01	–	–	–
1 flow, buffer 0.50 x BDP	reno	96.7	8.1	0.20	–	–	–
1 flow, buffer 1.00 x BDP	reno	100.0	22.3	0.07	–	–	–
1 flow, buffer 2.00 x BDP	reno	100.0	52.0	0.32	–	–	–
1 flow, buffer 0.10 x BDP	gentle-reno	92.7	0.7	0.01	–	–	–
1 flow, buffer 0.25 x BDP	gentle-reno	98.4	3.8	0.01	–	–	–
1 flow, buffer 0.43 x BDP	gentle-reno	100.0	9.2	0.01	–	–	–
1 flow, buffer 0.50 x BDP	gentle-reno	100.0	11.8	0.00	–	–	–
1 flow, buffer 1.00 x BDP	gentle-reno	100.0	29.0	0.00	–	–	–
1 flow, buffer 2.00 x BDP	gentle-reno	100.0	56.0	0.00	–	–	–
startup (first 10 s), 1 flow, buffer 1 x BDP	reno	–	–	–	172	–	–
startup (first 10 s), 1 flow, buffer 1 x BDP	gentle-reno	–	–	–	0	–	–
2 gentle-reno flows, 2nd joins at 10 s (40-60 s)	gentle-reno	–	–	–	–	25.0 / 25.0	1.00
gentle-reno vs reno, same path (20-60 s)	gentle-reno / reno	–	–	–	–	24.6 / 25.4	1.00

With small buffers, Gentle Reno keeps the link fuller than Reno (92.7 % vs. 82.0 % at 0.1 BDP; 100 % from 0.43 BDP, where Reno reaches 95 %), and its slow-start exit avoids the startup loss burst entirely (0 vs. 172 packets lost in the first 10 s). The cost is a higher average queue at the same buffer size (29 vs. 22 ms at 1 BDP), because its sawtooth never drops as far below the buffer limit.

Fairness and convergence. Two Gentle Reno flows with equal RTTs form an AIMD system: every loss multiplies both windows by 0.7 and so shrinks their difference by 30 %, while additive increase leaves the

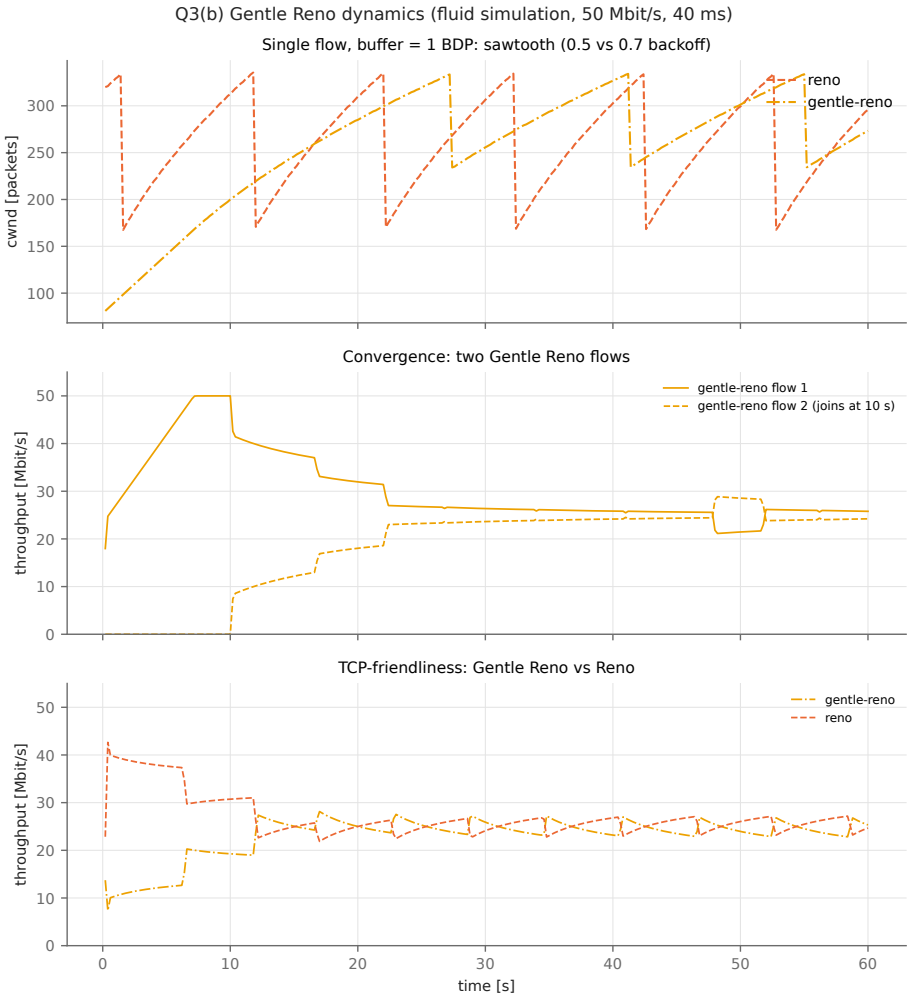


Figure 14: Top: single-flow sawtooth at 1 BDP of buffer. Middle: two Gentle Reno flows, the second joining at 10 s. Bottom: Gentle Reno and Reno sharing the bottleneck. *Generated by* `make_design_outputs`.

difference unchanged. By the Chiu–Jain argument the windows converge to an equal share and then oscillate around a stable operating point (a sawtooth between 0.7 and 1 times the window at which the buffer overflows). Convergence takes more loss events than for Reno (30 % of the difference removed per event instead of 50 %). In the simulation the second flow is within 15 % of the first about 14 s after joining, and over 40–60 s the split is 25.0/25.0 Mbit/s (Jain index 1.00). Against Reno, the choice of a predicts an even split, and the simulation gives 24.6/25.4 Mbit/s.

Assumptions. Losses are caused by congestion at a single drop-tail bottleneck; competing flows have similar RTTs; the receiver window does not limit the sender; the RTT signal is used only during slow start, so delay noise cannot cause cuts in steady state.

Limitations.

- As with Reno, throughput is inversely proportional to RTT, so flows with shorter RTTs get more bandwidth.
- In congestion avoidance it still fills the buffer until a loss, so on deep buffers it inflates RTT like CUBIC and Reno.
- Random loss still triggers cuts: on a path like Ningbo it would get about the same 0.2 Mbit/s as Reno, since its loss–throughput relation matches Reno's by design, far below BBR.
- With 0.5 packet/RTT growth it is even slower than Reno to use newly available capacity on high-BDP paths; CUBIC's time-based growth addresses exactly this, and a cubic increase function could be combined with $\beta = 0.7$ (which is what CUBIC does).
- It was evaluated only in a fluid simulation, which ignores ACK clocking, burstiness, delayed ACKs and SACK details, not as a kernel module on the measured paths.

5 Use of AI tools

We used Claude (Anthropic), through Claude Code, to analyse the measurement data (summary statistics, checking per-run anomalies such as receive-window limits in the raw samples) and to draft this report from our results. Every number in the report was checked against the generated tables and raw samples in the repository.

We also used AI to draft the scripts needed to run the experiments, which we then verified manually.